ÖVNINGSUPPGIFTER: LEKTION 2

Säkring, prestanda och djupanalys av databasrelationer

🟢 Gröna Uppgifter (Grundläggande SQL-relationer – Alla måste klara)

Uppgift 1
Den interna inventeringen. Skriv en INNER JOIN som listar alla aktiva servrar tillsammans med Username och Email på den användare som hanterar servern (ManagedByUserId).

Uppgift 2
Orkestreringsluckan. Skriv en LEFT JOIN som listar samtliga användare i databasen, oavsett om de hanterar en server eller inte. Sortera resultatet så att användare utan servrar hamnar överst.

Uppgift 3
De herrelösa maskinerna. Skriv en RIGHT JOIN som identifierar alla servrar som för närvarande saknar en tilldelad administratör (ManagedByUserId IS NULL).

Uppgift 4
Den totala överblicken. Skriv en FULL OUTER JOIN som kombinerar Users och Servers. Visa Username och ServerName för att kartlägga alla befintliga och saknade relationer i infrastrukturen.

Uppgift 5
Det fysiska låset. Skriv ett DDL-skript som lägger till en FOREIGN KEY-constraint på kolumnen ManagedByUserId i tabellen Servers så att den pekar på Id i Users. Sätt regeln till ON DELETE SET NULL.

🟡 Gula Uppgifter (Applikation, Prestanda & Parametrisering)

Uppgift 6
Miljökontrollen. Skriv en query som via en INNER JOIN hämtar användarnamn och servernamn, men filtrera resultatet så att du endast visar servrar som tillhör miljöerna 'Production' eller 'Staging'.

Uppgift 7
Snabba spår i loggen. Skapa ett NONCLUSTERED INDEX på tabellen Servers för kolumnen ManagedByUserId. Skriv en kort kommentar om varför detta är viktigt vid Foreign Key-valideringar.

Uppgift 8
SQLi-Labbet (Den sårbara koden). Skapa een C#-konsolapplikation i Visual Studio. Skriv en metod som tar en sökträng (e-post) via Console.ReadLine() och skickar den till databasen med en osäker, strängkonkaternerad ADO.NET-fråga.

Uppgift 9
Penetrationstestet. Kör din applikation från Uppgift 8. Mata in payloaden hacker@failsafe.it' OR '1'='1 och verifiera att din kod släpper igenom SQL-injektionen och dumpar tabellens innehåll.

Uppgift 10
Defend the Code (Parametrisering). Skriv om din C#-sökmetod från Uppgift 8. Implementera SqlParameter för att separera frågans struktur från datat. Verifiera att penetrationstestet från Uppgift 9 nu misslyckas.

🔴 Röda Uppgifter (Avancerad DevSecOps & Databashärdning)

Uppgift 11
Inkapsling via DDL. Skapa en Stored Procedure i SQL Server som heter sp_RegisterServer. Den ska ta emot parametrar för servernamn, IP-adress, miljö och hanterande användar-ID, och utföra en INSERT INTO.

Uppgift 12
Proceduranrop från C#. Uppdatera din C#-konsolapplikation så att du registrerar en ny server genom att anropa din Stored Procedure sp_RegisterServer via ADO.NET med CommandType.StoredProcedure.

Uppgift 13
RPC-Verifikationen. Kör ett penetrationstest mot ditt proceduranrop i C# genom att skicka in fientlig text i servernamnet. Dokumentera varför RPC-protokollet gör det omöjligt för strängen att exekveras som kod.

Uppgift 14
Principen om minsta behörighet (Least Privilege). Skriv ett DDL-skript i SSMS som skapar en ny SQL-inloggning och databasanvändare vid namn FailsafeApp. Blockera användaren från direktåtkomst till tabellerna med DENY SELECT, INSERT ON Users och DENY SELECT, INSERT ON Servers.

Uppgift 15

Skölden aktiveras. Ge din nya användare FailsafeApp exekveringsrättigheter (GRANT EXECUTE) på din Stored Procedure från Uppgift 11. Verifiera i SSMS att användaren kan köra proceduren men stoppas vid direkta SELECT-frågor.